THE ASSIGNMENT

Product listing page

Build a responsive product listing page in Next.js, using the supplied designs and JSON dataset.

Our new website will use Sitecore, Next.js and Vercel. This exercise assesses frontend implementation, a small backend API, and how you structure an application for content supplied by a headless CMS. You do not need access to Sitecore or to implement a CMS.

Expected effort: 2–4 hours
Use AI assistants, frameworks and libraries to work efficiently. Prioritise the responsive layout, API, filtering and sorting. We do not expect a production-ready deliverable; explain what remains unfinished.

What we will provide

- Desktop, tablet and mobile design references.
- A JSON dataset containing products, images, categories, technologies and measurement types.

Use our brand guidelines for typography, colours and the logo. These links are clickable in this PDF.

1. Implement the supplied design

Recreate the page as faithfully as reasonably possible, including the header and hero area, product cards, responsive layouts, display and filter controls, and alphabetical and popularity sorting.

Use the supplied JSON as the source of product content. Accommodate the dataset rather than hardcoding the products shown in the screenshots. The page should work across screen sizes, including widths between the supplied references.

Header navigation, login and site-wide search do not need working backend functionality. Product cards should link to the corresponding product URLs.

2. Make the listing functional

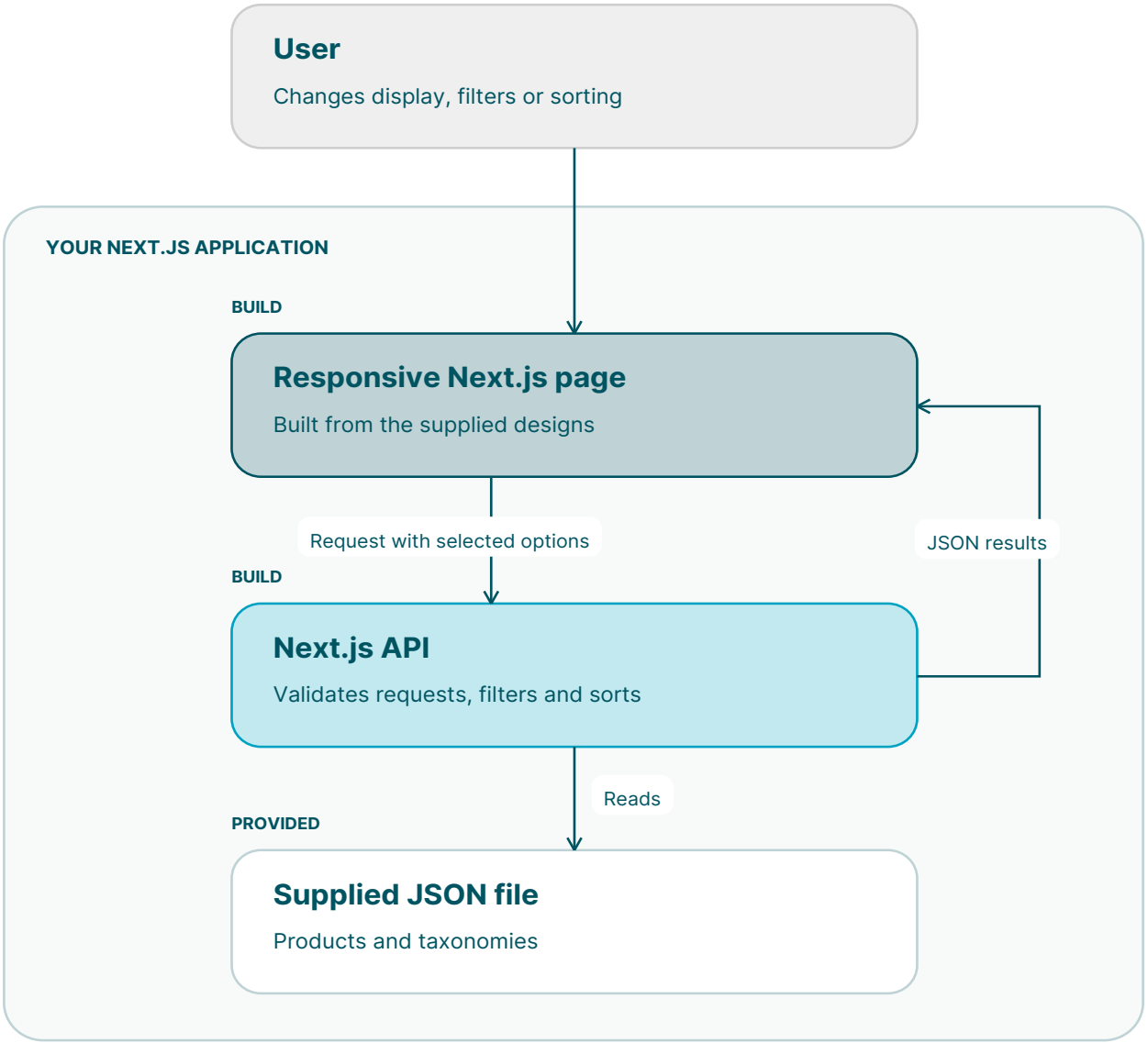
- Use the Display control to switch between products, categories, technologies and measurement types, using the corresponding JSON collections.
- Filter products by category and technology. These filters apply to the products view; use your judgement about controls in the other views.
- Switch between alphabetical and popularity sorting.
- Open and use the filters on mobile.

Consider how these controls work together and what users see when no results match.

ARCHITECTURE

What you need to build

Two connected deliverables: a responsive page and a working API.



Required flow: the frontend calls your API when filters or sorting change. The API performs product filtering and sorting on the server and returns the results for the frontend to display.

Both parts can be deployed together on Vercel. No separate backend service, database or Sitecore instance is required.

You choose the component structure, endpoints and response format. The diagram defines responsibilities, not a prescribed implementation.

IMPLEMENTATION

Behaviour and structure

Use your judgement
Where the designs or data leave room for interpretation, use your judgement and briefly document your assumptions. You may ask clarification questions. Any answer that materially changes the scope will be shared with all candidates in a written FAQ.

3. Build a small API

Create a read-only API within the Next.js application, using the supplied JSON as its data source.

- Accept query parameters for category, technology and sort order.
- Perform product filtering and sorting on the server.
- Return results as JSON.
- Handle invalid parameters deliberately and consistently.

Connect the product listing to this API so that changing filters or sorting updates the displayed results.

Choose and document your API structure and response format. Make the supporting data needed for the display options and filters available to the frontend.

A Next.js Route Handler is sufficient. No database, authentication, write operations or separate backend service is required.

4. Structure the implementation sensibly

Use reusable components and keep data access and listing logic separate from presentation, so the supplied JSON could later be replaced with a CMS or another API.

Choose an approach proportionate to this exercise. Consider:

- Semantic HTML and keyboard accessibility.
- Responsive layouts and image handling.
- Loading, empty and error states.
- Code clarity and maintainability.

No particular CSS framework or component library is required. If time permits, include a small number of meaningful automated checks covering filtering and sorting. Otherwise, explain how you would verify them.

5. Suggest an improvement

Identify one aspect of the design or interaction that you would improve. Briefly explain the problem, who would benefit, and your proposed solution.

Keep the supplied design as your primary implementation. A separate implementation or sketch of your improvement is optional.

SUBMISSION AND REVIEW

Show us your approach

AI and other tools

You may use AI assistants, documentation, libraries and other development tools.

Briefly describe how you used AI, if applicable. You should understand the submitted code and be prepared to explain and modify it.

Implement the supplied designs yourself. Do not copy HTML, CSS or JavaScript from the existing Malvern Panalytical website.

Time and priorities

We expect this exercise to take approximately 2–4 hours, depending on your approach and familiarity with the tools. Please feel free to use AI assistants, frameworks and libraries to help you work efficiently. We want to see how you approach the task without it taking up too much of your time.

Prioritise a faithful responsive layout and a working product listing backed by your API, including filtering and sorting. Use any remaining time to improve robustness, accessibility and verification.

We are not expecting a production-ready deliverable. Use your discretion to prioritise the work. State approximately how long you spent, anything unfinished, known issues or areas needing polish, and what you would improve with more time. Completing every detail is not a prerequisite for a strong submission.

What to submit

- A source repository with meaningful commits showing how your implementation developed.
- A working preview, preferably on Vercel.
- A short README covering setup, API usage, key decisions, assumptions, tool use and anything you would address with more time.
- Your proposed improvement.

Please commit as you progress rather than submitting everything as a single final commit. There is no required number or frequency of commits, and an untidy intermediate step is perfectly acceptable.

Follow-up discussion

We will ask you to walk us through selected parts of your implementation, explain your decisions and make a small change with us.

What we will assess

- Fidelity to the supplied designs.
- Responsive behaviour and functional correctness.
- API design and server-side logic.
- Component structure and separation of responsibilities.
- Accessibility and handling of edge cases.
- The judgement behind your assumptions and trade-offs.
- Your ability to explain, verify and adapt your implementation.